# Ders 3 Genişletilmiş Notu: RLE ile Görüntü Sıkıştırma, Bilgi Teorisi, Entropi ve Veri Modelleri

## Ders 3 Genişletilmiş Notu: RLE ile Görüntü Sıkıştırma, Bilgi Teorisi, Entropi ve Veri Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Geçen Haftanın Özeti: RLE ve MNP5
- 2. Görüntü Türleri ve Piksel Gösterimi
- 3. Bitmap Okuma, Scan Line ve Başlangıç Noktası
- 4. Siyah-Beyaz Resimde RLE
- 5. Renkli Palet Örneği: 660 Bitten Kaç Bite?
- 6. Kayıpsız ve Kayıplı RLE
- 7. RLE'nin Farklı Versiyonları
- 8. Tarama Biçimleri: Satır, Sütun, Rotate ve Zigzag
- 9. 8x8 Siyah-Beyaz Örnek ve RLE Özeti
- 10. RLE Ödevinin Akademik İçeriği
- 11. Bilgi Teorisi: Olasılık ve Bilgi
- 12. Entropi
- 13. Entropi, Ortalama Mesaj Uzunluğu ve Artıklık
- 14. Residual Sequence (Fark Dizisi)
- 15. Tekrar Eden Örüntüleri Blok Olarak Almak
- 16. Veri Modelleri: Fiziksel, Olasılık ve Markov
- 17. Markov Modelde Entropi
- Örnek Sorular


### 1. Geçen Haftanın Özeti: RLE ve MNP5

Hoca derse geçen hafta yarım kalan RLE (Run-Length Encoding) konusunu "görüntü sıkıştırma" bölümüyle tamamlayarak başladı. Geçen hafta RLE'nin daha çok resim sıkıştırmada başarılı olduğu, metinde başarılı olamadığı söylenmişti. Bunun sebebi metinlerde art arda gelen aynı karakter sayısının yetersiz olmasıdır; yalnız boşluk karakterlerinde bu tür tekrarlar olabilir.

RLE birçok sıkıştırma yönteminin ara adımıdır: JPEG resim formatında ve WinRAR, WinZip gibi araçlarda RLE bir ara adım olarak kullanılır.

RLE'de bir pikselin (ya da karakterin) kaç kez tekrar ettiği, ardından da değerin kendisi yazılır. `3 5` yazılmışsa değeri 5 olan piksel 3 kez tekrar ediyor demektir; bu bir karakter de olabilir (3 tane A). Ama yalnız sayı ve ardından değer yazmak yeterli değildir, çünkü okuyan taraf bir sayının run değeri mi yoksa bir piksel değeri mi olduğunu bilemez. Bunu ayırmak için önüne özel bir sembol konuyordu ve bilgi 3 byte ile temsil ediliyordu: özel sembol görülünce arkasından gelenin run değeri, onun arkasından gelenin de tekrar eden sembol olduğu anlaşılır. Resim dosyasında bu özel sembolün byte karşılığı bir renk olmayabilir, ama metin ya da exe dosyasında böyle bir karakter geçebilir. Bu yüzden RLE'nin ihtiyaca göre seçilen farklı versiyonları vardır.

Geçen hafta Microcom Networking Protocol'ün (MNP5) faks gibi uygulamalarda kullanılan versiyonu da anlatılmıştı: iki karakter art arda gelmişse olduğu gibi yazılır, üç tane gelmişse arkasına mutlaka run değeri yazılır ve özel sembole gerek kalmaz. Örneğin üç tane 25 art arda geliyorsa `25 25 25 0` yazılır; buradaki 0, üç tekrardan sonra ek tekrar olmadığını söyler. 0 yerine 5 yazılırsa 3 + 5 = 8 tane 25 art arda geliyor demektir.

## 2. Görüntü Türleri ve Piksel Gösterimi

Elimizdeki dosya karakterlerden değil piksellerden oluşuyorsa durum değişir (slayt "Görüntü Sıkıştırma", ekrandan, 00:10:00). Dijital bir görüntü piksel adı verilen noktalardan oluşur ve her piksel şu biçimlerden birinde olabilir:

- 1 bit B&W (siyah-beyaz): piksel 0 ya da 1'dir. 0 siyaha, 1 beyaza karşılık geliyorsa, siyah ve beyazın 8 bitlik kodu yerine tek bitle ifade edilir.
- Birkaç bit gray scale: siyah ile beyaz arasında değişen tonlar. 3 bitle 8 farklı kod yazılabilir; örneğin `000` siyah, `111` beyaz, `110` siyahın biraz açılmışı (füme), `001` beyaza yakın bir ton olabilir. 4 bitle 16 farklı kod olur.
- 8 bit color table: renkler 0 ile 255 arasındadır, 256 farklı renk vardır.
- 24 bit true color: red, green ve blue olmak üzere üç palet vardır; her biri 0-255 arasında değer alır ve 8 bit yer kaplar. $3 \times 8 = 24$ bit olduğu için bu resimlere true color denir.

Slayttaki tablo: 4 bit 16 renk, 7 bit 128 renk, 8 bit 256 renk.

Gri tonlu resimlerde de art arda tekrar eden değerler olabildiği için RLE'nin başarımı yüksek olabilir. 8 bitlik color table'da örneğin 25 hangi rengin koduysa art arda üç tane 25 gelebilir. Renk sayısı arttıkça değişkenlik artar ve art arda aynı rengin gelmesi zorlaşır. Siyah-beyaz resimde ise, resim özel olarak hazırlanmamışsa, bir siyah bir beyaz piksel diye giden bir yapı olmaz; art arda tekrar eden siyah ya da beyaz bloklar olacağı için RLE çok daha başarılıdır.

True color bir resimde üç paletin her biri ayrı ayrı kodlanır: red paletine ayrı, green'e ayrı, blue'ya ayrı RLE uygulanır. Açarken üçünden gelen sonuçlar bir araya getirilerek tek bir piksel olarak değerlendirilir. Hoca örnek olarak red paletinden 25., green'den 65., blue'dan 70. rengin alındığı bir pikseli anlattı; bu üç değerin birleşimi ekrana basılacak rengi verir.

Metin dosyasında RLE'nin başarısızlığını hoca slayttaki bir metin kutusuyla gösterdi: harfler art arda pek tekrar etmediği için RLE başarılı değildir.

## 3. Bitmap Okuma, Scan Line ve Başlangıç Noktası

Hoca slayta bir çiçek ve yaprak çizerek bu resmin sıkıştırılmak için nasıl okunması gerektiğini sordu. Görüntü üzerinde çalışanların bildiği gibi görüntüler scan line'lardan oluşur (ekrandan, 00:20:00). Scan line bir metin satırı gibi düşünülmemelidir: ekranda yazan "4 bit 16 renk" gibi tek bir metin satırı resim olarak görüntülense belki 6-7 scan line'dan oluşur. Bir satırın kaç scan line'a geldiğini seçilen format belirler.

Slayttaki maddeler:

- Pikseller bitmap içinde scan line'lara yerleşir.
- İlk bitmap pikseli resmin sol üst köşesindeki, son bitmap pikseli sağ alt köşesindeki noktadır.
- RLE ile bir resim sıkıştırılacaksa nereden başlanacağı önemlidir.
- Görüntü içinde rastgele seçilmiş bir pikselin komşularının kendisiyle aynı renkte olma ihtimali zayıftır.

Resim sol üstten başlanarak her scan line okunur ve sağ alt köşeye gelinceye kadar devam edilir. Okumanın stilleri de vardır: satır satır, kolon kolon ya da zigzag. Hangisinin daha etkili olduğu resme bağlıdır ve ileride örneklerle görüldü.

Hoca başlangıç noktasının önemini bir resim çizerek anlattı: bulutlar, deniz, ufuk çizgisi ve bir yelkenli. Gökyüzü mavi, bulutlar beyaz olsun. Rastgele bir yerden başlanırsa biraz mavi okunduktan sonra birden beyaza geçilir. Oysa en baştan başlansa gökyüzü uzun süre mavi okunur, sonra belli bir süre beyaz gelir, sonra yeniden maviye dönülür. İkinci örnek: içi boyalı, siyah dikey bir çizgi kolon kolon okunurken çizginin başından başlanırsa uzun süre siyah piksel gelir; rastgele bir noktadan başlanırsa çok az siyah pikselden sonra hemen beyaza geçilir ve art

arda tekrar eden renkler kaçırılır. Resmin içeriğine göre bir şey değişmeyebilir, ama değişme olasılığı yüksek olduğu için genelde sol üstten başlanır.

### 3.1 Neden Bitmap?

RLE ile sıkıştırılacak resim mutlaka bitmap (BMP) türünde olmalıdır. JPEG dosyası alınıp RLE ile sıkıştırılamaz, çünkü JPEG zaten sıkıştırılmış bir formattır; açıldığında görülen, sıkıştırılmış formatın açılmış hâlidir. TIFF dosyası da sıkıştırılmış bir resimdir. Bitmap, sıkıştırılmamış bir resmin en saf hâlidir.

Bitmap dosyasının formatı açıldığında başta bir header bulunur. Header resmin siyah-beyaz mı, gray scale mi olduğunu, piksellerin kaç bitle kodlandığını anlatır; arkasından data kısmı gelir. Data kısmını okumak kolaydır ve bitmap'i okuyup header ile data kısmını ayrı dizilere çeken hazır kütüphaneler kullanılabilir. RLE resmin bulunduğu data kısmına uygulanır; header sıkıştırılmış verinin başına eklenir, çünkü header olmadan sıkıştırılmış veri açılamaz.

## 4. Siyah-Beyaz Resimde RLE

Slayt başlığı: "Compressor bitmap'i satır satır, sütun sütun veya zigzag olarak tarar" (ekrandan, 00:27:30). Örnekte elimizde siyah-beyaz bir resim vardır.

- Compressor ilk okunacak pikselin beyaz olduğunu kabul eder.
- İlk piksel beyaz değilse output stream'e 0 adet beyaz olarak yerleşir.

Output stream'in başında 0 görülüyorsa anlamı "0 tane beyaz"dır. Arkasından 7 gelmişse 7 tane siyah, sonra 5 gelmişse 5 tane beyaz vardır. İlk okunan run beyaz, sonrakiler sırayla siyah, beyaz, siyah… olarak yerleşir. Siyah-beyaz resimde renk kodu yazmaya gerek kalmaz; yalnız run uzunlukları yazılır.

Slayttaki örnek: Resim 17 beyaz, onu izleyen 1 siyah ve 55 beyaz pikselden oluşmuşsa output stream'e `17 1 55` diye yerleştirilir. Açarken önce 17 beyaz, sonra 1 siyah, sonra 55 beyaz yazılır.

Output stream'in başına bitmap'in çözünürlük değeri de yazılır. Hocanın örneğinde başa 72 ve 256 yazılmıştır: 72 scan line okunacak ve her scan line'da 256 piksel vardır. Çözücü run'ları topladıkça ilerler: 17 beyaz, 1 siyah (toplam 18), 55 beyaz (toplam 73). Toplam henüz 256 olmadığı için birinci scan line dolmamıştır; 256 tamamlanınca ikinci scan line'a geçilir.

Dekompres için resmin satır satır mı, kolon kolon mu, zigzag mı okunduğunun da bilinmesi gerekir. Hoca bu bilginin header'da tutulabileceğini söyledi: örneğin `00` satır satır, `01` kolon kolon, `10` ya da `11` zigzag okunduğunu gösterebilir.

### 4.1 Şeklin Karmaşıklığı ve Sıkıştırma Oranı

Sıkıştırılmış bir stream'in uzunluğu şeklin karmaşıklığına bağlıdır: şekilde ne kadar fazla detay varsa sıkıştırma oranı o kadar kötüdür. Siyah-beyaz bile olsa bir odadan çekilmiş fotoğraf karmaşıktır; oysa beyaz zemin üzerinde büyük siyah bir artı işareti karmaşık değildir ve daha iyi sıkıştırılır.

Uniform bir alanda RLE için sıkıştırma oranı kabaca şöyledir:

$$\text{oran} \approx \frac{\text{şeklin çevre uzunluğu}/2}{\text{alan içindeki toplam piksel sayısı}}$$

Çevre uzunluğu scan line'lardan bulunur. Örneğin şeklin üzerinden 50 scan line geçmişse her scan line şekle bir noktadan girer, bir noktadan çıkar. Slayttaki ifade: "Her scan line için bir giriş bir çıkış noktası olduğundan $2\times$ scan line değeri uniform şeklin çevre uzunluğunu verir." Yani $50 \times 2 = 100$ çevre uzunluğudur. Bunun yarısı (yani scan line sayısı) şeklin içindeki piksel sayısına bölünürse RLE'nin sıkıştırma oranı yaklaşık olarak bulunur.

## 5. Renkli Palet Örneği: 660 Bitten Kaç Bite?

Hoca bir kitaptan alınmış görüntü üzerinden ilerledi (ekrandan, 00:30:00). Solda 1'den 16'ya numaralı 16 renkli bir palet var (`000000` siyah, `0000FF` mavi, `F86C0F` turuncu, `00FF00` yeşil, `008C00` koyu yeşil, `0C4317` çok koyu yeşil vb.). 16 renk $2^4$ olduğu için her renk 4 bitle kodlanır; renklerin kendisi 8 bit ya da daha az bitle ifade edilmiş olabilir, konu bu değildir.

Resim 15 piksel genişlik, 11 piksel yüksekliktedir. Matristeki her kare bir pikseldir ve her piksel 16 renkten birini alır. Sıkıştırmasız boyut:

$$15 \times 11 \times 4 = 660 \text{ bit}$$

### 5.1 Run Değeri + Renk Kodu ile Sıkıştırma

Kullanılan RLE versiyonu her run için önce run değerini, sonra rengin kendisini yazar. Birinci scan line'da 15 tane 5 numaralı renk (turuncu) vardır: `15 5`. İkinci scan line'da 7 tane 5, 4 tane 11, 4 tane 5 vardır: `7 5 4 11 4 5` (ekrandan, 00:34:25).

Hoca kazanç ve kaybı şöyle anlattı: tek bir piksel olan yerde 1 değer yerine 2 değer (run ve renk) yazılır, kayıp olur. Ama 15 aynı pikselin olduğu yerde 15 değer yerine 2 değer yazılır, kazanç olur. Bazı yerlerde kazanç, bazı yerlerde zarar vardır ve bunlar bir şekilde dengelenir.

Sıkıştırılmış hâlde 134 değer vardır. Her biri 4 bitle kodlanır:

$$134 \times 4 = 536 \text{ bit}$$

Run değerlerinin de 4 bitle yazılmasının sebebi, resim 15 piksel genişlikte olduğu için yazılabilecek en büyük run değerinin 15 olmasıdır; bu da 4 bite sığar. Run değeri 8 bitle tutuluyor ve öyle kodlanıyorsa bit sayısı değişir. Hoca bu yüzden elde edilen başarının resmin içeriğine ve kullanılan RLE versiyonuna bağlı olduğunu vurguladı.

Sıkıştırılmış dosyanın açılabilmesi için nasıl sıkıştırıldığının header'da olması gerekir. Örneğin ilk iki byte resmin eni ve boyu, üçüncü byte run için ve renk kodu için kaç bit ayrıldığı, bir başka byte da satır satır mı, kolon kolon mu, zigzag mı okunduğu olabilir. Açma işlemi bu bilgilerle yapılır.

### 5.2 Renk Sayısı Azalınca ve Artınca

Aynı boyutta, renk sayısı azaltılmış bir resimde (turuncu, koyu yeşil ve siyah ağırlıklı) aynı yöntemle 86 değer elde edilir (ekrandan, 00:36:40):

$$86 \times 4 = 344 \text{ bit}$$

Renk sayısı azaldığı için bir satırda daha çok tekrar eden renk vardır. Önceki resimde aralara başka renkler girdiği için her biri 2 değer ekliyordu. Hoca, resmin siyah-beyaz olduğu ya da bütün yeşillerin turuncu olduğu düşünülürse bu sayının daha da küçüleceğini, bu yüzden siyah-beyaz resimlerde RLE'nin çok daha iyi performans vereceğini söyledi.

Tersine, renk sayısı artırılmış bir resimde dış kenar bir turuncu bir sarı diye gider, içeride açık mavi, koyu mavi, yeşil ve koyu yeşil vardır; yalnız ortadaki siyahlar değişmemiştir. Bu durumda 311 değer çıkar (ekrandan, 00:38:10):

$$311 \times 4 = 1244 \text{ bit}$$

Sıkıştırmadan önce 660 bit olan resim 1244 bite çıkmıştır; bu sıkıştırma değil genişletmedir.

> **Ders sonrası not:** Hoca bu noktada (00:38:30) "siyah beyaz resimlerde RL kullanmam abesle iştigal" dedi. Bağlam ve slayt, renklerin sürekli değiştiği karışık resimlerde RLE kullanmanın anlamsız olduğunu gösteriyor; hoca hemen önce siyah-beyaz resimlerde RLE'nin çok iyi sonuç verdiğini söylemişti.

### 5.3 JPEG'de RLE Neden İşe Yarıyor?

**Soru (hocanın kendi sorduğu):** Siyah-beyaz bir JPEG pek görmeyiz; JPEG'in bir ara adımında RLE kullanılıyorsa nasıl başarılı oluyor?

**Cevap:** JPEG, RLE'yi özgün renklere uygulamaz. Önce renkler üzerinde bir dönüşüm uygulanır, değerler belirli bir sayı aralığına indirgenir ve RLE bu aralıktaki değerlere uygulanır. Böylece sanki daha az renkli bir resme RLE uygulanmış gibi olur. Açarken önce RLE çözülür, sonra uygulanan dönüşümün tersi alınır ve özgün veriye yakın ya da birebir aynısı elde edilir.

> **Ek bilgi (derste anlatılmadı):** JPEG'de 8x8 bloklara büyük olasılıkla ayrık kosinüs dönüşümü (DCT) uygulanır ve katsayılar kuantalanır. Kuantalamadan sonra yüksek frekanslı katsayıların çoğu büyük olasılıkla sıfır olduğu için, katsayılar zigzag sırayla okunduğunda uzun sıfır dizileri oluşur ve RLE bu dizileri verimli kodlar.

## 6. Kayıpsız ve Kayıplı RLE

Hoca RLE'nin hem kayıplı hem kayıpsız bir yöntem olabileceğini söyledi. Şimdiye kadar anlatılanların hiçbirinde kayıp yoktur.

Resimde bir piksellik farklı bir renk olduğunu ve bu rengin çevresindeki renklerle arasındaki farkın çok çok az olduğunu düşünelim; örneğin çevredeki renkler 15, bu piksel 14 olsun. Göz bu farkı algılayamaz. Bu piksel 15 yapılırsa run uzar, ama kayıplı sıkıştırma yapılmış olur. Gözün algılayamayacağı noktalardaki kayıplar resimde önemli değildir.

> **Hocanın vurgusu:** Tıbbi bir görüntüde kayıplı sıkıştırma yapılamaz. Çok küçük bir nokta, örneğin küçük bir kist, kayıplı sıkıştırmada kaybolabilir ve bu ciddi sonuçlara yol açabilir.

Kayıplı kullanımın bir biçimi kısa run'ları ihmal etmektir. Hoca, JPEG formatında kaydederken run değerinin sorulabildiğini, örneğin 2'den küçük run'ların ihmal edilip komşu pikselin renginin verilebildiğini anlattı; bu durumda kayıplı görüntü sıkıştırma yapılmış olur.

> **Ders sonrası not:** Yaygın resim düzenleyicilerde JPEG kaydederken sorulan ayar büyük olasılıkla kalite (quality) ayarıdır; kısa run'ları ihmal etme fikri ise kayıplı RLE'nin genel mantığını anlatır.

## 7. RLE'nin Farklı Versiyonları

Hoca RLE ödevinde istenen versiyonun seçilebileceğini, bu yüzden versiyonları bilmek gerektiğini söyledi (slayt "RLE Farklı Versiyonları", ekrandan, 00:40:00).

Örnek: RLE algoritmasını gray scale resimlerin sıkıştırılmasında kullanalım (run değeri, piksel renk kodu). 8 bitlik gray scale bir bitmap'te $2^8 = 256$ farklı renk olabilir, yani renkler 0-255 arasındadır.

```
Uncompressed: 12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1 ......
Compressed:   9 12 35 76 112 67 3 87 6 5 1 ...
```

Art arda 9 tane 12 olduğu için `9 12` yazılır; 35, 76, 112, 67 tek oldukları için olduğu gibi kalır; 3 tane 87 için `3 87`, 6 tane 5 için `6 5` yazılır. Slaytta run değerleri maviyle gösterilmiştir. İnsan maviye bakarak bunların run olduğunu bilir, ama bilgisayara bunu anlatmak gerekir. Bilgisayar bu diziyi "9 tane 12, 35 tane 76, 112 tane 67, 3 tane 87, 6 tane 5…" diye yanlış okuyabilir. Problem, tekrar sayısının algoritmaya nasıl anlatılacağıdır; bunun için farklı versiyonlar ortaya atılmıştır.

### 7.1 Versiyon I-a: Soldaki Biti Flag Yapmak

Slayt: "Görüntü 128 (0-127) gray scale ile sınırlı ise, her byte'ın soldaki biti gray scale değeri mi? tekrar sayısı mı? diye kullanılır." (ekrandan, 00:50:00)

Resim 8 değil 7 bitlikse renkler 0-127 arasındadır ve bir byte'ın en soldaki 8. biti hiçbir zaman rengin kodu olmaz. Bu bit flag olarak kullanılır: örneğin soldaki bit 0 ise geri kalan 7 bit run değeri, 1 ise renk kodu olarak okunur. Hangisinin 0, hangisinin 1 olacağına kodlayan kişi karar

verir; dekompresörü de aynı kişi yazdığı için ona göre açar. Aynı fikir 4 bitlik, 3 bitlik resimlere de uygulanabilir.

### 7.2 Versiyon I-b: 255'i Escape Kodu Yapmak

Slayt: "Eğer gray scale değeri 256 (0-255) ise bu değer bir azaltılarak kullanılır (255 renk değeri 254 olarak alınır). 255 sayısı toplam değerden önce yazılarak, arkasından gelen iki byte'ın tekrar sayısı ve pixel değeri olacağı bilgisi gönderilir."

Renkler 0-255 arasındaysa 255. renk 254. renk olarak kodlanır; bu iki renk birbirine çok benzediği için önemli değildir. 255 sayısı ilk öğretilen yöntemdeki özel sembol gibi escape kodu olur.

```
Uncompressed: 12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1 ......
Compressed:   255 9 12 35 76 112 67 255 3 87 255 6 5 1 ...
```

Çözücü şöyle okur: 255 geldi, bundan sonraki 9 run, 12 tekrar eden değer. Sonra 255 gelmemiş, 35 renk kodu; 76 ve 112 ve 67 de renk kodu. 255 geldi, 3 run, 87 değer. 255 geldi, 6 run, 5 değer. 1 renk kodu.

### 7.3 Versiyon II: 8 Byte için Bir Flag Byte'ı

Slayt: "Her bir byte'ın ne olduğunu gösteren bir bit kullanılır. Bu extra bitler 8'lik gruplar hâlinde tutulur. Output stream'e tekrar ve renk değerleri gönderilmeden önce bu bilgiye ait flag değerlerini tutan 1 byte'lık bu değer gönderilir. Her 8 byte için fazladan 1 byte gönderilir."

```
Uncompressed: 12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1 ......
Compressed:   10000010 9 12 35 76 112 67 3 87 100.... 6 5 1 ...
```

Önce bir byte okunur. Bu byte'ın bitleri, ardından gelen 8 byte'ın her biri için run mı yoksa özgün renk mi olduğunu söyler: 1 ise run değeri, 0 ise özgün renk. `10000010` byte'ı: 9 run, 12 renk, 35 renk, 76 renk, 112 renk, 67 renk, 3 run, 87 renk demektir. Bu 8 byte bitince yeni bir flag byte'ı (`100...`) okunur ve sonraki 8 byte onun bitlerine göre yorumlanır.

Bazı uygulamalar işi abartıp byte yerine word kullanır ve arkasından gelen 16 byte'ı o word'e göre yorumlar. Ne kadar kazandırıp kaybettirdiği farklı veriler üzerinde denenerek bulunur; hocaya göre özellikle 0-255 arasındaki renklerde byte almak daha anlamlıdır.

### 7.4 Versiyon III: True Color Resim

Slayt: "Renkli bir resimde her pixel 3 byte ile gösterilir. Red, Green ve Blue." (ekrandan, 00:51:20)

```
4 pixel değerimiz:
(171 85 34) (172 85 35) (172 85 30) (173 85 33)

Her palet için ayrı ayrı RLE uygulanır:
(171 172 172 173)
(85 85 85 85)
(34 35 30 33)
```

24 bit 24 bit olarak ele alınmaz, 8-8-8 (red, green, blue) olarak ele alınır. Önce bütün piksellerin red değerleri alınıp RLE uygulanır, red'in bittiği yerden green paleti, green'in bittiği yerden blue başlar. Red'in ilk scan line'ını, sonra blue'nun scan line'ını birbirine bağlamak gibi karmaşık bir yola girilmemelidir: çözmek zorlaşır, hata olasılığı artar. Her palet bitirilip sonra diğerine geçilir. Örnekte green dizisi `85 85 85 85` tek bir run olarak çok iyi sıkışır.

Siyah-beyaz resimde ise run değerini ayırt etmeye bile gerek yoktur: ilk okunan beyaz kabul edilir, sonrakiler sırayla siyah, beyaz diye gider (Bölüm 4).

## 8. Tarama Biçimleri: Satır, Sütun, Rotate ve Zigzag

Slayt: "RLE için resim dosyaları satır-satır, sütun-sütun veya zig-zag olarak okunur." (ekrandan, 01:00:00)

### 8.1 F Harfi Örneği

Slaytta 16 piksel genişliğinde, ızgara üzerinde siyah bir F harfi var. Üst çubuk 10. piksele kadar, F'nin gövdesi 2 piksel genişlikte. Satır satır okumada soldan başlanır ve sırayla birinci, ikinci, üçüncü scan line okunur. İlk piksel beyaz olmadığı için önce 0 yazılır:

```
Satır satır (tahtadaki yazım):
0 10 6
0 10 6
0 2 ...
```

Birinci satırda 0 beyaz, 10 siyah, 6 beyaz; ikinci satırda yine 0 beyaz, 10 siyah, 6 beyaz; üçüncü satırda 0 beyaz, 2 siyah ve geri kalanı beyazdır.

Aynı resim sütun sütun okunursa: birinci sütunda beyaz yok, 12 siyah var; ikinci sütunda yine 0 beyaz, 12 siyah; üçüncü sütunda 0 beyaz, 2 siyah, 3 beyaz, 2 siyah, 6 beyaz:

```
Sütun sütun (tahtadaki yazım):
0 12 0 12 0 2 3 2 6 ...
```

Hangisinin daha iyi olduğu ancak ikisi de denenerek görülür. Hoca bir başka resim çizdi: iki kalın dikey siyah çizgi. Bu resmi sütun sütun okumak daha uzun siyah run'lar verdiği için daha iyi sıkıştırır. Karar resmin içeriğine göre verilir.

### 8.2 Rotate Okuma

Satır satır ve sütun sütun okumanın bir de rotate (dönerek) hâli vardır. Hoca tahtaya "Rotate satır, Rotate sütun" yazdı. Rotate'de satır sonuna gelince satır başına dönülmez; alttaki satırdan geriye doğru devam edilir (yılan gibi).

F örneğinde rotate satır okuma: birinci satır `0 10 6` ile biter. Rotate'de ikinci satıra sağdan girilir; ikinci satırın sonunda da 6 beyaz olduğu için beyazlar devam eder ve 6 + 6 = 12 beyaz olur. Sonra 10 siyah gelir; satır sonunda aşağı dönülünce üçüncü satırın başındaki 2 siyah da eklenir ve 12 siyah olur. Böylece daha büyük bloklar alınır.

Bunun sebebi piksellerin komşuluğudur: bir pikselin kendine bitişik pikselle ilişkisi, resmin uzak bir köşesindeki pikselle ilişkisinden daha fazladır. Uç uca komşu piksellerin renkleri birbirine daha yakın ve uyumlu olur. Bir pikselin 8 komşusu vardır ve bunlar resmin başındaki bir pikselden daha ilişkilidir. Bu yüzden satır satır da, sütun sütun da rotate okunursa RLE'nin sıkıştırma performansı daha yüksek olur.

### 8.3 Zigzag Okuma

Hocaya göre en ideal okuma zigzag okumadır. 8x8 bir ızgarada zigzag okumada ilk piksel okunur, hemen yanındakine geçilir, sonra çapraz aşağı, sonra bir alta, sonra çapraz yukarı… diye köşegenler boyunca ilerlenir (slayttaki 8x8 ızgara, ekrandan, 01:00:00). Amaç komşulukları kullanmaktır.

JPEG, RLE ile birlikte zigzag okuma kullanır. Ama resmin tamamını zigzag okumaz, çünkü bütün resmi zigzag okumak istenen kazancı sağlamaz. Resmi 8x8 ya da 16x16'lık parçalara böler ve her parçayı kendi içinde zigzag okur. Bu küçük parçalardaki pikseller birbirine daha çok ilişkilidir. Yani 8x8 = 64 piksellik bloklara zigzag uygulanır, sonra sonraki 64 piksellik bloğa geçilir ve her bloğa RLE uygulanır.

## 9. 8x8 Siyah-Beyaz Örnek ve RLE Özeti

Slayttaki 8x8 siyah-beyaz resim satır satır (row-row) ve sütun sütun (column-column) okunmuştur (ekrandan, 01:02:40). Resmin satırları (B beyaz, S siyah):

```
1. satır: B S B S S B S B
2. satır: B S B S S B S B
3. satır: B S B S S B S B
4. satır: B S B S S B S B
5. satır: B B B B B B B S
```

```
6. satır: S S S S S S S S
7. satır: B B B B B B B S
8. satır: S S S S S S S S
```

Satır satır okumada ilk okunan 1 beyaz, 1 siyah, 1 beyaz, 2 siyah, 1 beyaz, 1 siyah, 1 beyazdır ve satır sonuna `<eoln>` (end of line) eklenir. End of line da bir byte sayılır. Örnekte her değerin 1 byte olduğu kabul edilmiştir:

```
Row-row
1 1 1 2 1 1 1 <eoln>   -> 8 x 4 = 32
7 1 <eoln>             -> 3 x 2 = 6
0 8 <eoln>             -> 3 x 2 = 6
                          44 bytes

Column-column
5 1 1 1 <eoln>         -> 5 x 3 = 15
0 4 1 1 1 1 <eoln>     -> 7 x 4 = 28
4 4 <eoln>             -> 3 x 1 = 3
                          44 bytes
```

Birinci tür satırdan 4 tane olduğu için 8 byte 4 ile çarpılır (32 byte). “7 beyaz, 1 siyah, eoln” satırı 3 byte’tır ve 2 tane vardır; “0 beyaz, 8 siyah, eoln” satırı da 3 byte’tır ve 2 tane vardır. Toplam 44 byte.

> **Ders sonrası not:** Sütun sütun okumanın toplamı $15+28+3 = 46$ byte eder; slaytta yazan 44 byte büyük olasılıkla bir yazım hatasıdır. Sütunlar tek tek sayıldığında 1., 3. ve 6. sütun `5 1 1 1`, 2., 4., 5. ve 7. sütun `0 4 1 1 1 1`, 8. sütun `4 4` verir (python ile kontrol edildi). Hoca anlatımda da “kolon kolon okudum, 44 byte çıktı” dedi.

Hoca, run değerlerinin burada byte olarak alındığını, ama kaç bitle tutulacağının değerlere bağlı olduğunu söyledi: run 127’ye çıkabiliyorsa 1-2 bitte tutulamaz. Siyah-beyaz resimde renk için 1 bit, 3 bitlik gray scale’de 3 bit ayrılır; bu durumda hesaplar değişir. Rotate okunsaydı satır ya da sütun bazındaki değerler de değişebilirdi.

### 9.1 RLE Özeti

Slayt “Özet” (ekrandan, 01:10:00):

- RLE kayıpsız bir sıkıştırma yöntemidir (istenirse kayıplı da kullanılabilir).
- Aynı değere sahip uzun verilerde en yüksek başarıyı verir. Hoca örnek olarak kendi ekranının görüntüsünü verdi: art arda çok fazla beyaz piksel olduğu için RLE çok başarılı olur.
- Kodlaması kolaydır.
- Çeşitli varyasyonları vardır.
- Hızlı çalışır; çok hızlı compress ve decompress eder.
- Karışık veri üzerinde negatif sıkıştırma yapar: her yerde farklı karakterler olan bir veride özgün hâlinden daha fazla bit elde edilir, sıkıştırma değil genişletme yapılmış olur.

Metin dosyalarında başarısız, görüntülerde başarılıdır.

## 10. RLE Ödevinin Akademik İçeriği

Hoca RLE ile ilgili bir ödev verdi. Akademik içeriği şöyledir:

- Bitmap formatının header’ında hangi bilgilerin tutulduğuna kısaca bakılır; formatı çözmek için hazır kütüphane kullanılabilir.
- Birden fazla (örneğin iki) siyah-beyaz, `.bmp` uzantılı resim alınır. Gray scale ya da renkli resim istenmiyor.
- Data kısmı okunur, kendi yazılan RLE koduyla sıkıştırılır ve açılır. RLE’yi yapan hazır bir kod kullanılmaz. Sıkıştırılıp açılan veri özgün resmi vermiyorsa bir problem vardır; veriyorsa doğru sıkıştırılmıştır.
- Header kısmı sıkıştırılmaz, ama boyutu sonuca eklenir. Hocanın örneği: rotate row okumayla veri 64 byte’a iniyor, header 32 byte, toplam 96 byte; özgünü 250 byte olan dosya

96 byte'a düşmüş olur. Aynı veri rotate column ile 94, zigzag ile 99 byte gibi farklı sonuçlar verebilir.
- Özgün boyut, rotate row, rotate column ve zigzag sonuçları bir tabloda karşılaştırılır ve altına açıklama yazılır. Zigzag tüm resim üzerinde gezdirilebilir ya da resim örneğin dört parçaya bölünüp her parçaya uygulanabilir.
- Resimler birbirinden farklı seçilir; amaç hangi okuma biçiminin hangi resimde daha iyi sıkıştırdığını ya da fark olup olmadığını görmektir.
- Kullanılan RLE versiyonu rapora yazılır; farklı versiyonların sonuçları karşılaştırılabilir.

## 11. Bilgi Teorisi: Olasılık ve Bilgi

Aradan sonra yeni bir konuya, bilgi teorisi (information theory) başlığı altında entropiye geçildi. Bilgi teorisi başlı başına bir dönemlik ders olarak açılan bir konudur; hoca yalnız entropi kavramının sıkıştırmada nerede ve ne amaçla kullanıldığını anlattı.

Slayt "Bilgi Teorisi" (ekrandan, 01:40:00): Tabiata baktığımızda bir olayın gerçekleşme ihtimali ne kadar düşükse o olayın taşıdığı bilgi o kadar fazladır. Tersi de aynı şekilde düşünülebilir: bir olayın gerçekleşme ihtimali ne kadar fazlaysa o olay o kadar az bilgi içerir. Olasılık ile bilgi arasında ters orantı vardır.

Hocanın örnekleri:

- Yazın 28-30 derecede deniz kenarında güneşlenirken "bugün kar yağar mı?" diye sorulur. İhtimali sıfıra yakındır. Ama kar yağarsa bu olay çok önemli hâle gelir ve taşıdığı bilgi çok fazladır. (Slayttaki örnek: sıfır derecede karın yağması çok olasıdır ve fazla bilgi taşımaz; 20 derecede kar yağarsa olağanüstü bir durum vardır ve bu olay fazla bilgi taşır.)
- Hava kapalı, bir haftadır yağmur yağıyor, sabah da yağıyor. "Öğlen yağmur yağar mı?" Elbette yağacak; olasılığı çok yüksek olduğu için bir bilgi içermez.

### 11.1 Öz Bilgi Formülü

$P(A)$, $A$ olayının gerçekleşme ihtimali; $I(A)$, $A$ olayının taşıdığı bilgi (information) olsun. Bilgi olasılıkla ters orantılı olduğu için $1/P(A)$ alınır ve logaritması hesaplanır:

$$I(A) = \log_x \frac{1}{P(A)} = -\log_x P(A)$$

$P(A)$ paydan yukarı $P(A)^{-1}$ olarak geçtiği için logaritmanın önüne eksi gelir. Taban $x$, bilgisayarda bir şey ya vardır ya yoktur, yani iki durum olduğu için 2'dir. Hoca, bilginin teraziye konup "3 kilo geldi" diye ölçülemeyeceğini; olasılık 0 ile 1 arasında sayısal ifade edilebildiği gibi bilginin de sayısal ifade edilmesi gerektiğini ve bu yüzden olasılığa bağlı tanımlandığını söyledi. Olasılığın tersinin logaritması, o olayın kaç bitle kodlanacağını gösterir.

### 11.2 Sıkıştırmadaki Anlamı

Bir sayfada A harfinin mi J harfinin mi daha çok geçtiği sorulsa saymadan A denir. A'nın olasılığı yüksek, taşıdığı bilgi azdır; J'nin olasılığı düşük, taşıdığı bilgi yüksektir. ASCII tablosunda A da J de 8 bitle kodlanır. Sıkıştırma yapılacaksa bilgisi düşük A daha az bitle, bilgisi yüksek J daha fazla bitle ifade edilmelidir.

Hocanın sayısal örneği (slayttaki tahta notları, ekrandan, 01:40:00):

- Metinde 20 tane A ve 5 tane J var. Hepsi 8 bitle kodlanırsa A'lar $20 \times 8 = 160$ bit, J'ler $5 \times 8 = 40$ bit eder.
- A 3 bitle kodlanırsa $20 \times 3 = 60$ bit olur (160'tan 60'a iner).
- J 12 bitle kodlanırsa $5 \times 12 = 60$ bit olur (40'tan 60'a çıkar).
- J'de kaybedilen 20 bit, A'da kazanılan 100 bitten azdır. Toplam 200 bit yerine 120 bit kullanılır (ara adım eklendi), yani bir sıkıştırma yapılmış olur.

### 11.3 Para Atma Örneği

Hilesiz bir para atıldığında yazı ve tura gelme olasılığı $1/2$'dir. Bilgi:

$$I = -\log_2 \frac{1}{2} = 1 \text{ bit}$$

Logaritma negatif çıkar, önündeki eksiyle pozitif 1 olur. Hilesiz parayı kodlamak için 1 bit yeterlidir: tura 0, yazı 1 ya da tersi.

Hileli bir parada yazı gelme olasılığı $1/8$, tura gelme olasılığı $7/8$ olsun:

$$I(\text{yazı}) = -\log_2 \frac{1}{8} = -\log_2 2^{-3} = 3 \text{ bit}$$

$$I(\text{tura}) = -\log_2 \frac{7}{8} \approx 0{,}193 \text{ bit}$$

Bunlar teorik değerlerdir. Pratikte 0,5 ya da 0,7 bit diye bir şey yoktur; bitler 1, 2, 3 diye gider. Pratikte tura için en az 1 bit, yazı için 3 bit kodlama gerekir.

## 12. Entropi

Tek bir olay (bir para atışı) yerine birden fazla bağımsız olay olabilir: para bir kez, ikinci kez, üçüncü kez atılır. Bağımsızlık, her atışın bir öncekiyle hiçbir ilgisi olmaması, önceki bilgiyi kullanmaması demektir. $n$ tane olay $A_1, A_2, \ldots, A_n$ olsun; bütün olaylardan oluşan kümeye $S = \bigcup_{i=1}^{n} A_i$ denir.

Bu örnek seti içindeki ortalama öz bilgiye (self-information) entropi denir ve $H$ ile gösterilir. Hoca kullandığı kitapta $H$ kullanıldığı için $H$'yi seçtiğini, bazı kaynaklarda farklı harfle görülebileceğini söyledi. Her olayın olasılığı ile o olayın bilgisinin çarpımlarının toplamı olay setinin ortalama bilgisini verir:

$$H = \sum_i P(A_i) I(A_i) = -\sum_i P(A_i) \log_2 P(A_i)$$

Bilgi doğrudan ölçülemediği için yerine $-\log_2 P(A_i)$ konur. Olasılık 0 ile 1 arasında olduğundan logaritma negatif çıkar; öndeki eksiyle çarpılınca entropi her zaman pozitif olur.

Olaylar kodlanırken entropi, olması gereken ideal ortalama binary sembol uzunluğunu verir. Örneğin bir metindeki 29 karakterin olasılıkları ve bilgileri çıkarılır; A'nın gelmesi bir olay, B'nin gelmesi bir olay, 29 karakterin tamamı $S$ kümesini oluşturur. Olasılık ve bilgi çarpımları toplandığında entropi örneğin 5 çıkarsa bu "5 bit/sembol" demektir. Sembol, kod verilen birimdir: bir karakter de olabilir, iki karakter de.

ASCII'de her sembol için ortalama bit uzunluğu 8'dir. Ama metindeki karakterler aynı olasılıkla geçmez; olasılıkları farklıysa bilgileri de farklıdır ve hepsini 8 bitle kodlamak sıkıştırma yapmamak demektir. Hangi yöntem kullanılırsa kullanılsın, elde edilen ortalama uzunluk hesaplanan entropiye yakın olmalıdır.

Slayt (ekrandan, 01:50:00): "Shannon göstermiştir ki en iyi kayıpsız sıkıştırmada kodlanan sembollerin ortalama bit uzunluğu teoride bulduğumuz kaynağın entropy'sine eşit olmalıdır. Ancak, bu ideal sıkıştırma da mümkün olmamaktadır." Örneğin entropi 2,71 çıkarsa bu teorik değerdir; pratikte semboller 1, 2, 3, 4 bitle kodlanır ve ortalamaları buna yakın olmalıdır.

> **Ders sonrası not:** Hoca "Shannon Fano diye bir adam var; Shannon adı, Fano soyadı" dedi. Shannon-Fano kodlaması büyük olasılıkla iki ayrı kişinin, Claude Shannon ile Robert Fano'nun adını taşır. Hoca dersin sonunda ilk kodlama yönteminin Shannon-Fano olacağını söyledi.

**Ek bilgi (derste anlatılmadı):** Shannon'ın kaynak kodlama teoremine göre büyük olasılıkla herhangi bir kayıpsız önek kodun ortalama uzunluğu $L$ için $L \geq H$ olur ve uygun bir kodla $H \leq L < H+1$ sağlanabilir. Semboller bloklar hâlinde kodlandıkça sembol başına ortalama uzunluk büyük olasılıkla entropiye daha çok yaklaşır.

### 12.1 Dört Sembollü Örnek

Slayt (ekrandan, 01:52:50): Kaynak alfabesi $a_1, a_2, a_3$ ve $a_4$'ten oluşan bir set olup sembollerin olasılıkları $a_1 = 0{,}49$, $a_2 = 0{,}25$, $a_3 = 0{,}25$ ve $a_4 = 0{,}01$ olsun. Metinde en çok $a_1$, en az $a_4$ vardır.

$$H = -(0{,}49 \log_2 0{,}49 + 0{,}25 \log_2 0{,}25 + 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) \approx 1{,}57$$

Bu set ortalama 1,57 bitle kodlanabilir. Bu teoriktir; 1,57 bit olmaz, ama bazı semboller 1, bazıları 2, bazıları 3 bitle kodlanır (örneğin 1 bitlik 10 tane, 2 bitlik 5 tane, 3 bitlik 2 tane) ve ortalamaları 1,57'ye yakın olmalıdır. Python ile $H = 1{,}5707$ bulundu.

## 13. Entropi, Ortalama Mesaj Uzunluğu ve Artıklık

Aynı dört sembol pratikte nasıl kodlanır? Dört sembol olduğu için $2^2 = 4$ gerekir; her sembol 2 bitle kodlanır: `00`, `01`, `10`, `11`. Bu kodlamadaki ortalama mesaj uzunluğuna $S$ denir. Teori 1,57, pratik 2 der. Aradaki mutlak fark redundancy'dir (artıklık); ilk haftada fazlalık olarak bahsedilmişti:

$$R = |S - H| = |2 - 1{,}57| = 0{,}43 \text{ bit}$$

Sıkıştırılmış dosyada hâlâ 0,43 bitlik bir fazlalık vardır. $R$ sıfıra ne kadar yakınsa o kadar ideal bir sıkıştırma yapılmış olur. Hoca bir karşılaştırma verdi: bir yöntemle $R = 2{,}43$, başka bir yöntemle $R = 0{,}43$ çıkarsa ikincisi çok daha iyidir ve onunla devam edilir. Gerçek dünyada $R$'nin sıfıra ulaşması zordur; sayılarla oynayarak $R$'yi sıfır yapan örnek verilebilir, ama amaç farkı olabildiğince küçültmektir.

### 13.1 2 Tabanında Logaritma

Slayt "Not: 2 tabanında logaritma nasıl alınır?" (ekrandan, 02:00:00). Hesap makineleri 10 tabanında (ya da doğal) logaritma alır:

$$\log_2 x = q \;\Rightarrow\; x = 2^q \;\Rightarrow\; \ln x = q \ln 2 \;\Rightarrow\; q = \frac{\ln x}{\ln 2} \;\Rightarrow\; \log_2 x = \frac{\log_{10} x}{\log_{10} 2}$$

### 13.2 Sayı Dizisi Örneği

Slayt (ekrandan, 02:00:00): `1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10` şeklinde bir sayı dizisi verilmiş olsun. Bunlar renk kodları ya da bir metin olabilir. Bu sayı dizisini kodlayacak olursak ortalama bit uzunluğu ne olur?

Dizide 16 sayı ve 10 farklı sembol vardır (1'den 10'a). 1, 6, 7 ve 10 birer kez; 2, 3, 4, 5, 8 ve 9 ikişer kez geçer:

$$P(1) = P(6) = P(7) = P(10) = \frac{1}{16}, \quad P(2) = P(3) = P(4) = P(5) = P(8) = P(9) = \frac{2}{16}$$

Hoca slaytta $P(1) = 1/6$ yazıldığını görüp bunun $1/16$ olması gerektiğini söyledi ve slaytta düzeltilmesini istedi.

$$H = -\left(4 \cdot \frac{1}{16} \log_2 \frac{1}{16} + 6 \cdot \frac{2}{16} \log_2 \frac{2}{16}\right) = 4 \cdot \frac{1}{16} \cdot 4 + 6 \cdot \frac{1}{8} \cdot 3 = 1 + 2{,}25 = 3{,}25 \text{ bit}$$

Her sembol ortalama 3,25 bitle kodlanmalıdır; bazıları 1, bazıları 2, bazıları 3 bit olabilir. 10 farklı sembol sabit uzunlukla 4 bitle kodlanırsa $S = 4$ ve $R = 4 - 3{,}25 = 0{,}75$ olur. Bu sıfıra çok yakın değildir; daha iyi bir kodlama seçilebilir.

## 14. Residual Sequence (Fark Dizisi)

Aynı veriyle daha etkili bir sıkıştırma için veri residual sequence'e dönüştürülebilir (ekrandan, 02:10:00). İlk eleman sabit kalır, sonra ardışık elemanlar arasındaki farklar alınır: $2 - 1 = 1$, $3 - 2 = 1$, $2 - 3 = -1$, $3 - 2 = 1$, $4 - 3 = 1$, ...

```
Orijinal:  1 2 3  2 3 4 5  4 5 6 7 8 9  8 9 10
Residual:  1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1  1
```

Değerler belli bir şeyin çevresinde dönen sayılardır. Residual diziden özgün diziye dönülebilir: 1 ile başlanır; 1'e 1 eklenir 2, 2'ye 1 eklenir 3, 3'ten 1 çıkarılır 2, 2'ye 1 eklenir 3... Verinin taşıdığı bilgi değişmez, yalnız başka bir forma geçirilir ve özgün hâle geri dönülebilir.

10 farklı sembol yerine artık yalnız 2 sembol vardır: 1 ve $-1$. 16 değerin 13'ü 1, 3'ü $-1$'dir:

$$P(1) = \frac{13}{16}, \quad P(-1) = \frac{3}{16}, \quad H = -\left(\frac{13}{16}\log_2\frac{13}{16} + \frac{3}{16}\log_2\frac{3}{16}\right) \approx 0{,}70 \text{ bit}$$

Python ile $H = 0{,}696$ bulundu. Bu dizi 1 bitle rahatça kodlanır: bit 0 ise 1, bit 1 ise $-1$. $S = 1$ ve $R = |1 - 0{,}70| = 0{,}3$ olur; sıfıra yakındır, bu kodlama yapılabilir.

### 14.1 Alıcı Özgün Veriye Nasıl Döner?

Slayt: "Alıcı bu diziden orijinal veriye dönüş yapabilir mi? Sekansın modelinin çıkarılması gerekir." $x_n$ özgün dizinin $n$. elemanı, $r_n$ residual dizinin $n$. elemanı olmak üzere model:

$$x_n = x_{n-1} + r_n$$

Kodlanan dizinin özgünle doğrudan ilişkisi yoktur; karşı tarafın bunun bir dönüşüm olduğunu bilmesi gerekir. Bunun için header'da bir bilgi tutulur: örneğin 17. byte 0 ise veri özgün veridir, 1 ise residual fark alınmış bir dizidir ve ilk terim başa yazılmıştır. Alıcı ilk terimi referans alıp ekleyerek ya da çıkararak özgün veriyi elde eder. Hoca bunu bir şifre gibi anlattı: karşı taraf modeli önceden bilir; sıkıştırılmış veri açılınca residual dizi elde edilir ve modele sokularak özgün veriye ulaşılır.

Hocanın adım adım çözümü: $x_0$ yok sayılır (0), $x_1 = 0 + r_1 = 1$; $x_2 = x_1 + r_2 = 1 + 1 = 2$; $x_3 = x_2 + r_3 = 2 + 1 = 3$; $x_4 = x_3 + r_4 = 3 + (-1) = 2$...

### 14.2 Ne Zaman İşe Yarar?

Bu bir dönüşümdür: veri sıkıştırılmadan önce sıkıştırmaya daha uygun hâle getirilir. JPEG'de RLE'den önce uygulanan dönüşüm de aynı amaca hizmet eder. Her uygulamada başarılı olmaz; sayılar arasındaki geçişlerin çok küçük değerler olması gerekir. Hocanın örneği: bir odanın sıcaklığı 10 dakikada bir ölçülür. Normal şartlarda değerler 40,2; 40,1; 40; 40,2; 41 gibi çok yakın değerlerle ufak oynamalar gösterir. Bu tür veride residual sequence çok daha başarılı sonuç verir: 10 farklı sembol yerine 2 sembole inilir.

Bu modellere statik model denir: parametreler $n$ değiştikçe değişmez; model her indis için aynıdır. Bazı dinamik modellerde model $n$'nin değerine göre değişir (dizideki değişim değerlerine göre farklı modeller kullanılır). Dinamik modeller sıkıştırma ve açma zamanı açısından daha maliyetlidir.

## 15. Tekrar Eden Örüntüleri Blok Olarak Almak

Hoca bunun bir vize sorusu olduğunu söyledi (ekrandan, 02:17:50). Slayt: 1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2 şeklinde verilen sayı dizisini ortalama kaç bit uzunluğunda kodlayabiliriz?

Çoğu öğrencinin çözümü: 3 farklı sembol (1, 2, 3) vardır. 20 sembolde 5 tane 1, 5 tane 2, 10 tane 3 vardır:

$$P(1) = P(2) = \frac{5}{20} = \frac{1}{4}, \quad P(3) = \frac{10}{20} = \frac{1}{2}$$

$$H = -\left(\frac{1}{4}\log_2\frac{1}{4} + \frac{1}{4}\log_2\frac{1}{4} + \frac{1}{2}\log_2\frac{1}{2}\right) = 0{,}5 + 0{,}5 + 0{,}5 = 1{,}5 \text{ bit/sembol}$$

Slaytta kırmızıyla: "Sekans 20 sembolden oluştuğu için kodlamak için gerekli bit sayısı 30." 3 sembol için 2 bit gerekir; hepsi 2 bitle ya da bazıları 1, bazıları 2 bitle kodlanabilir ve $S$ ile $H$ farkı bulunur.

Az sayıda öğrenci ise veri setindeki örüntülere (pattern) baktı. Slayttaki soru: "Aynı sekansta blokları ikişer sembolden oluşacak şekilde alırsak sonuç değişir mi?" İkişerli bakıldığında yalnız 12 ve 33 örüntüleri vardır. Üçerli alınsa 121, 233, 333, 123, 331, 312 gibi çok daha fazla farklı sembol çıkar ve iş karışır. İkişerli alındığında 12 5 kez, 33 5 kez geçer; entropi formülüne konunca:

$$H = 1 \text{ bit/sembol (blok)}$$

12 blokları 0, 33 blokları 1 ile kodlanırsa $S = 1$ olur; $R = |1 - 1| = 0$. Yapılabilecek en iyi kodlama yapılmış, artıklık sıfır olmuştur.

> **Ders sonrası not:** Slaytta ve anlatımda $P(12) = P(33) = 1/4$ ("5 bölü 20") yazıyor. İkişerli bloklarla dizi 10 bloktan oluşur ve her blok 5 kez geçer; doğrusu $P(12) = P(33) = 5/10 = 1/2$'dir. $H = 1$ bit/blok sonucu bu olasılıklarla çıkar ve 20 sembollük dizi 10 bitle kodlanır (python ile kontrol edildi).

Hoca bu örnekle şunu özetledi: elde onlarca sıkıştırma yöntemi vardır ve seçim verinin tipine göre değişir: resim mi, metin mi, ses mi; resimse kaç bitlik; metinde matematiksel ifadeler, sayılar mı daha fazla, belirli örüntüler var mı? Hem metin hem görüntü içeren bir doküman, metin ve görüntü ayrılıp her birine ayrı yöntem uygulanırsa daha iyi sonuç verir.

## 16. Veri Modelleri: Fiziksel, Olasılık ve Markov

Slayt "Modeller" (ekrandan, 02:20:00): 1- Fiziksel Model, 2- Olasılık Model, 3- Markov Model. Verinin modelini iyi veya doğru bir şekilde oluşturmak, kaynağın entropisini tahmin etmek ve etkili bir sıkıştırma algoritması seçmek için önemlidir.

### 16.1 Fiziksel Model

Verinin fiziksel özelliği kullanılır. Hocanın örnekleri: bir ses gürültülü bir ortamda mı, sessiz bir ortamda mı alındı? Gürültülü ortamda alındıysa sıkıştırmadan önce gürültü kaldırılmalıdır; gürültülü resim için de aynısı geçerlidir. Bir köydeki hanelerin elektrik ya da su faturalandırması günün saat dilimine (gece, gündüz) ya da evde yaşayan nüfusa göre yapılır; bunlar fiziksel modellerdir. Bir şeyin sıcaklığını ölçüp sonuçları elde etmek ve residual modele geçmek de aslında fiziksel bir modeldir.

### 16.2 Olasılık Modeli

İstatistiksel bir modeldir: kaynak tarafından üretilen her sembolün birbirinden bağımsız ve eşit olasılıkla üretildiği kabul edilir. Bu modele ignorance model denir ve kaynak hakkında hiçbir bilgi yoksa kullanılır.

Hoca masasındaki eşyalarla anlattı: mouse, defter, zil, kâğıt tutacağı, su bardağı gibi 10 farklı nesne var. Bir mouse varsa olasılığı $1/10$, ikinci bir kablolu mouse da varsa $2/10$ olur. Masada laptop olduğu için mouse olması ya da su olduğu için bardak olması gibi bağımlılıklar kullanılmaz. Metinde A'dan önce ya da sonra B gelmesine bağlı bir olasılık hesaplanmaz. Piksel de karakter de nesne de olsa olasılıklar bağımsız üretiliyorsa olasılık modelidir.

Ignorance modelde olasılıklar yerine sembollerin kullanım sıklıklarına göre olasılık hesaplanıyorsa buna probability model denir: her sembolün metinde geçme sayısına bakılarak olasılığı çıkarılır; resimde her piksel ayrı bir nesnedir. Hoca ilk haftalarda görülecek yöntemlerin hep olasılıksal model olacağını söyledi.

### 16.3 Markov Modeli

Slayt "Markov Model" (ekrandan, 02:30:00): Sembollerin olasılıkları birbirlerine bağımlı olarak hesaplanır (adını A.A. Markov Rus matematikçiden alır). Kayıpsız sıkıştırma yöntemleri için kullanılır. "Discrete Time Markov Chain" olarak da bilinir.

Masada mouse pad duruyorsa mouse'un olasılığı farklı, yalnız bilgisayar duruyorsa farklı, yanında cep telefonu duruyorsa farklıdır. Metinde A'nın olasılığı öncesinde B, C, D ya da J gelmesine göre farklıdır; 29 farklı karaktere bağlı olarak hesaplanmış olasılıklar vardır.

$\{x_n\}$ gözlemlenen bir sekans olsun. Bu sekans $k$. dereceden ($k^{th}$-order) Markov modeli olarak şöyle gösterilir:

$$P(x_n \mid x_{n-1}, \dots, x_{n-k}) \approx P(x_n \mid x_{n-1}, \dots, x_{n-k}, \dots)$$

Geçmişteki $k$ sembol bilgisi, işlemin geçmişteki tüm bilgisine yaklaşık olarak eşittir. A'nın olasılığı kendinden önceki 2 karaktere bağlıysa $k = 2$, 5 karaktere bağlıysa $k = 5$'tir; $k$ teoride $n$'ye kadar gidebilir. Hoca $n = 3$ ve $k = 2$ için koşulun $x_2, x_1$'e kadar gittiğini tahtada gösterdi.

Pratikte büyük $k$ çok maliyetlidir. Kendinden önceki 1000 karaktere bakılarak olasılık hesaplanırsa ilk 1000 karakter hiç kodlanamaz, olduğu gibi yazılır ve çok sayıda koşullu olasılık değeri gerekir. Genelde $k$ en fazla 2 ya da 1 seçilir; sıkıştırmada genelde 1 ya da 2 görülür.

Bu yaklaşıklığın sebebi, geçmişe bir sembol daha eklendiğinde olasılığın çok az değişmesidir. Hocanın örneği (tahta, ekrandan, 02:30:00): A'nın tek başına olasılığı 0,5 olsun. Kendinden önce B gelen durumlarda ($ba$) $P(a \mid b) = 0{,}7$, önce C gelen durumlarda ($ca$) $P(a \mid c) = 0{,}6$ olsun. İki karakterlik geçmiş (`bd` ardından `a`) alınınca 0,69 civarı bir değer çıkar. Geçmiş uzatıldıkça değişimler küçülür; bu yüzden $k$ küçük tutulur.

## 17. Markov Modelde Entropi

Bağımsız olasılık kullanılırken entropi, olayın olasılığı çarpı olayın bilgisi ($-\log_2$ olasılık) toplamıyla hesaplanıyordu. Bağımlı olasılık kullanılınca formül değişir.

Siyah-beyaz bir resimde iki durum vardır: white state ($S_w$) ve black state ($S_b$). Beyazdayken sonraki piksel ya beyaz ya siyah olur:

- Beyazdayken beyaz gelirse aynı durumda kalınır; olasılığı $P(w \mid w)$.
- Beyazdayken siyah gelirse siyah durumuna geçilir; olasılığı $P(b \mid w)$.
- Siyahtayken siyah gelirse aynı durumda kalınır; olasılığı $P(b \mid b)$.
- Siyahtayken beyaz gelirse beyaz durumuna geçilir; olasılığı $P(w \mid b)$.

Artık beyazın olasılığı kendinden önce gelen piksele bağlıdır. Her durumdan çıkan iki olasılığın toplamı 1'dir, bu yüzden biri bilinirse diğeri $1-$ o değer olarak bulunur.

Sistemin entropisi, durumların olasılığı ile durumların entropisinin çarpımlarının toplamıdır:

$$H = \sum_i P(S_i) H(S_i)$$

$$H(S_b) = -\left(P(w \mid b) \log_2 P(w \mid b) + P(b \mid b) \log_2 P(b \mid b)\right)$$

$$H(S_w) = -\left(P(b \mid w) \log_2 P(b \mid w) + P(w \mid w) \log_2 P(w \mid w)\right)$$

İki durum olduğu için toplam 1'den 2'ye gider.

### 17.1 Örnek

Slayt (ekrandan, 02:40:00): $P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w/b) = 0{,}3$, $P(b/w) = 0{,}01$ olarak verilmiş olsun. Hem Markov model ile hem de olasılıksal model ile sistemin entropisini hesaplayınız. Model birinci derecedir: her piksel kendinden önceki bir piksele bağlıdır.

Verilmeyen olasılıklar tamamlanır: $P(b \mid b) = 1 - 0{,}3 = 0{,}7$ ve $P(w \mid w) = 1 - 0{,}01 = 0{,}99$.

Olasılıksal model (bağımsız):

$$H = -(0{,}8 \log_2 0{,}8 + 0{,}2 \log_2 0{,}2) = 0{,}722 \text{ bit}$$

Markov model:

$$H(S_b) = -(0{,}3 \log_2 0{,}3 + 0{,}7 \log_2 0{,}7) = 0{,}881 \text{ bit}$$

$$H(S_w) = -(0{,}01 \log_2 0{,}01 + 0{,}99 \log_2 0{,}99) = 0{,}081 \text{ bit}$$

$$H = 0{,}2 \times 0{,}881 + 0{,}8 \times 0{,}081 = 0{,}241 \text{ bit}$$

Değerler python ile doğrulandı.

> **Ders sonrası not:** Hoca örneği anlatırken "Hidden Markov Model kullanıyorum" dedi. Örnekteki durumlar (siyah/beyaz) doğrudan gözlendiği için bu, gizli (hidden) Markov modeli değil, büyük olasılıkla birinci dereceden sıradan bir Markov modelidir. Ayrıca anlatımda $P(b \mid w)$ "0,1", $P(w \mid w)$ önce "0,09" sonra "0,99" diye geçiyor ve iki durumun entropisi sözlü olarak "0,81" diye okunuyor; doğru değerler slayttaki $P(b \mid w) = 0{,}01$, $P(w \mid w) = 0{,}99$, $H(S_b) = 0{,}881$ ve $H(S_w) = 0{,}081$'dir.

Markov modelin entropisi (0,241) olasılıksal modelinkinden (0,722) küçüktür. Teoride 0,24 ya da 0,72 bit yoktur; iki sembollü bu örnekte pratikte yine 1 bitle kodlanır. Ama 29 karakterden oluşan bir metin ya da 30 farklı renk gibi çok sembollü bir kaynakta bu entropi farkları farklı bit uzunluklarına yansır. Entropinin küçük olması daha ideal bir kodlama yapılabileceğini, yani Markov modelin daha iyi sıkıştırma performansı vereceğini gösterir.

Ama olasılıksal modeli kodlamak daha kolay ve hızlıdır; Markov model daha karmaşıktır ama daha iyi sonuç verir. Sembol sayısı da önemlidir; denemeler yapılarak hangi modelin daha etkili olduğu bulunur.

## Örnek Sorular

#### Soru 1 (ekrandan, 00:34:25)

15 x 11 piksellik, 16 renkli paletle çizilmiş bir resim run değeri + renk kodu versiyonuyla RLE'ye sokuluyor. Sıkıştırmasız ve sıkıştırılmış boyutlar nedir?

**Çözüm (derste verildi):** 16 renk için $\log_2 16 = 4$ bit gerekir. Sıkıştırmasız: $15 \times 11 \times 4 = 660$ bit. Sıkıştırılmış hâlde 134 değer (run ve renk) vardır; run değerleri en fazla 15 olduğu için onlar da 4 bitle yazılır: $134 \times 4 = 536$ bit. Renkleri azaltılmış resimde $86 \times 4 = 344$ bit, renkleri sürekli değişen resimde $311 \times 4 = 1244$ bit (genişletme).

#### Soru 2 (ekrandan, 00:27:30)

Siyah-beyaz bir resim 17 beyaz, onu izleyen 1 siyah ve 55 beyaz pikselden oluşuyor. Output stream'e ne yazılır? İlk piksel siyah olsaydı ne değişirdi?

**Çözüm (derste verildi):** `17 1 55`. İlk piksel beyaz kabul edildiği için renk kodu yazılmaz. İlk piksel siyah olsaydı stream 0 ile başlardı (0 adet beyaz), örneğin `0 7 5 ...` = 0 beyaz, 7 siyah, 5 beyaz.

**Soru 3 (ekrandan, 00:50:00)**

`12 12 12 12 12 12 12 12 12 35 76 112 67 87 87 87 5 5 5 5 5 5 1` dizisini 255'i escape kodu olarak kullanan RLE versiyonuyla ve 8 byte için bir flag byte'ı kullanan versiyonla sıkıştırınız.

**Çözüm (derste verildi):**

```
255 escape:   255 9 12 35 76 112 67 255 3 87 255 6 5 1 ...
Flag byte:    10000010 9 12 35 76 112 67 3 87 100.... 6 5 1 ...
```

Escape versiyonunda 255 rengi 254 olarak kodlanır. Flag versiyonunda `10000010`, ardından gelen 8 byte'tan 1. ve 7.'sinin (9 ve 3) run, diğerlerinin renk olduğunu söyler.

**Soru 4 (ekrandan, 01:02:40)**

Bölüm 9'daki 8x8 siyah-beyaz resmi satır satır ve sütun sütun RLE ile kodlayınız (her değer ve `<eoln>` 1 byte).

**Çözüm (derste verildi, düzeltmeyle):** Satır satır: $8 \times 4 + 3 \times 2 + 3 \times 2 = 44$ byte. Sütun sütun: $5 \times 3 + 7 \times 4 + 3 \times 1 = 46$ byte. Slaytta sütun toplamı 44 yazıyor; python ile yapılan sayımda 46 byte çıktı (Bölüm 9'daki ders sonrası not).

**Soru 5 (ekrandan, 01:52:50)**

Sembol olasılıkları $a_1 = 0{,}49$, $a_2 = 0{,}25$, $a_3 = 0{,}25$, $a_4 = 0{,}01$ olan setin entropisi ve 2 bitlik sabit kodlamadaki artıklık nedir?

**Çözüm (derste verildi):** $H \approx 1{,}57$ bit/sembol, $S = 2$, $R = 0{,}43$ bit.

**Soru 6 (ekrandan, 02:00:00 ve 02:10:00)**

`1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10` sayı dizisini kodlarsak ortalama bit uzunluğu ne olur? Bu diziden bir residual sequence oluşturursak dizi ortalama kaç bit uzunluğunda kodlanabilir? Alıcı özgün veriye dönebilir mi?

**Çözüm (derste verildi):** Özgün dizide $H = 3{,}25$ bit; 4 bitlik sabit kodla $R = 0{,}75$. Residual dizi `1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1`, $P(1) = 13/16$, $P(-1) = 3/16$, $H \approx 0{,}70$ bit; 1 bitlik kodla $R = 0{,}3$. Alıcı $x_n = x_{n-1} + r_n$ modelini biliyorsa özgün veriye döner.

**Soru 7 (ekrandan, 02:17:50; hocanın belirttiğine göre bir vize sorusu)**

`1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2` şeklinde verilen sayı dizisini ortalama kaç bit uzunluğunda kodlayabiliriz? Aynı sekansta blokları ikişer sembolden oluşacak şekilde alırsak sonuç değişir mi?

**Çözüm (derste verildi, düzeltmeyle):** Tek tek: $P(1) = P(2) = 1/4$, $P(3) = 1/2$, $H = 1{,}5$ bit/sembol, 20 sembol için 30 bit. İkişerli bloklar: 10 blok, `12` ve `33` beşer kez, $P(12) = P(33) = 1/2$ (slaytta $1/4$ yazıyor), $H = 1$ bit/blok. `12` → 0, `33` → 1 ile $S = 1$, $R = 0$; dizi 10 bitle kodlanır.

**Soru 8 (ekrandan, 02:40:00)**

$P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w/b) = 0{,}3$, $P(b/w) = 0{,}01$ olarak verilmiş olsun. Hem Markov model ile hem de olasılıksal model ile sistemin entropisini hesaplayınız.

**Çözüm (derste verildi):** Olasılıksal model $H = 0{,}722$ bit. Markov model: $H(S_b) = 0{,}881$, $H(S_w) = 0{,}081$, $H = 0{,}2 \times 0{,}881 + 0{,}8 \times 0{,}081 = 0{,}241$ bit.

**Soru 9 (derste sözlü, 01:42:06)**

Yazı gelme olasılığı $1/8$, tura gelme olasılığı $7/8$ olan hileli bir parada yazı ve turanın bilgisi kaç bittir?

**Çözüm (derste verildi):** $I(\text{yazı}) = -\log_2(1/8) = 3$ bit, $I(\text{tura}) = -\log_2(7/8) \approx 0{,}193$ bit. Pratikte tura en az 1 bit, yazı 3 bitle kodlanır.

## Dersin Toparlaması

Dersin ilk yarısında RLE'nin görüntü sıkıştırmadaki kullanımı işlendi. Siyah-beyaz, gri tonlu, color table ve true color görüntülerin piksel gösterimi, bitmap'in header ve data kısımları, scan line kavramı ve okumanın sol üstten başlamasının önemi anlatıldı. Siyah-beyaz resimde ilk run'ın beyaz kabul edildiği RLE, 16 renkli 15x11 resim üzerinden renk sayısının sıkıştırmaya etkisi (660 bitten 536, 344 ya da 1244 bite), run değerini renkten ayırmak için kullanılan versiyonlar (soldaki bit flag, 255 escape, flag byte'ı, true color'da paletlerin ayrı kodlanması) ve satır, sütun, rotate ve zigzag tarama biçimleri karşılaştırıldı. RLE'nin kayıpsız olduğu, istenirse kısa run'lar ihmal edilerek kayıplı kullanılabileceği ve karışık veride negatif sıkıştırma yaptığı vurgulandı.

İkinci yarıda bilgi teorisine geçildi. Olasılık ile bilgi arasındaki ters orantı $I(A) = -\log_2 P(A)$ ile ifade edildi; entropi ideal ortalama bit uzunluğu, artıklık ise ortalama mesaj uzunluğu ile entropi arasındaki fark olarak tanımlandı. Residual sequence ve tekrar eden örüntüleri blok alma örnekleri, veriyi dönüştürmenin entropiyi nasıl düşürebileceğini gösterdi. Son olarak fiziksel, olasılık ve Markov modelleri anlatıldı ve Markov modelin aynı kaynak için daha düşük entropi verdiği bir örnekle gösterildi. Hoca, model seçiminin yer ve zaman arasındaki bir tercih olduğunu söyledi ve bir sonraki konunun Shannon-Fano kodlaması olacağını belirtti.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.